

Clock Tree Synthesis — Project Report

Wong Cheuk Yin
1155192671

May 2, 2026

1 Introduction

Clock Tree Synthesis (CTS) is a key physical design task that distributes the clock signal from source taps to all sink pins with balanced delay. In this project, the routing space is modeled as a 2D grid with edge capacity limits and rectangular blockages, so the solver must satisfy multiple constraints while still achieving good timing quality. The optimization target follows the provided evaluator: minimizing global skew (maximum sink delay minus minimum sink delay) together with total routed wirelength.

Our implementation uses a two-stage pipeline. The first stage is **pin-to-tap assignment** (MFMC stage), where each pin is assigned to exactly one tap under `MAX_LOAD` constraints, followed by local refinements to reduce outliers and satellite-like assignments. The second stage is **A* routing**, where each tap builds a rectilinear tree to connect its assigned pins while respecting blockage legality and global edge `CAPACITY`. To improve robustness against local minima, we run multiple routing retries with diversified pin/tap orders and keep the best valid solution.

This report presents the problem formulation, algorithm design, implementation details, and experimental analysis of the proposed CTS solver.

2 Problem Formulation

2.1 Input Model

The input is a square routing grid with a set of clock taps, a set of clock pins, and a set of rectangular routing blockages. The key parameters are:

- `MAX_LOAD`: maximum number of pins that can be driven by each tap.
- `GRID_SIZE`: width and height of the square routing area.
- `CAPACITY`: edge capacity limit of each grid edge.
- `PINS numPins`: pin count and each pin coordinate `PIN index x y`.
- `TAPS numTaps`: tap count and each tap coordinate `TAP index x y`.
- `BLKS numBlks`: blockage count and each rectangle `BLK index xll yll xur yur`.

2.2 Output Format

The output specifies assignment and routing of each pin to one tap. For each tap, the format is:

- `TAP index`

- `PINS num`
- a list of driven pins: `PIN index`
- `ROUTING numEdges`
- a list of routed unit edges: `EDGE x y x' y'`

This block is repeated for all taps.

2.3 Evaluation Metric

The project objective is to generate a feasible clock tree while minimizing delay imbalance and wirelength. The cost function in the specification is:

$$\text{cost} = (\max d_i - \min d_i) \cdot \text{numTaps} + \sum_k \text{TreeLength}(\text{TAP}_k)$$

where d_i is the delay (path length) of pin p_i , `numTaps` is the number of taps, and `TreeLength(TAPk)` is the total length of the rectilinear tree of tap k .

2.4 Constraints

The generated solution must satisfy all constraints stated in the project specification:

1. No short or open pins.
2. The number of pins driven by each tap cannot exceed `MAX_LOAD`.
3. The number of routing edges across each grid edge cannot exceed `CAPACITY`.
4. Routing edges cannot be placed across blockage interiors (routing on blockage boundaries is allowed).

3 Implementation Details

3.1 Pin-to-Tap Assignment (Greedy + Discount + Swap)

The assignment module is implemented in `mfmc.cpp/hpp`. We use a three-step refinement pipeline:

1. **Greedy assignment:** build all pin-tap candidate edges with Manhattan cost, sort them, and assign each pin to the cheapest feasible tap under `MAX_LOAD`.
2. **Discount-based reassignment:** detect split-cluster/outlier taps from per-tap maximum assigned distance; then reduce the corresponding tap costs (discount factor) and rebuild assignment by greedy matching. This process is repeated for a bounded number of iterations.
3. **Swap refinement:** run best-partner 2-swap to locally improve assignment. Candidate source pins are selected from high-risk taps (gated by mean/std of max distance), and each source pin considers top destination taps. A swap is applied only when net gain is positive.

We firstly implement a plain greedy assignment algorithm in the early stage. then we found that in some test cases, the result show splited clusters in one tap (see Figure 1). It is probably because the algorithm is weak against grid with concentrated taps. To tackle this problem, we implement a discount-based reassignment algorithm. We detect the split-cluster/outlier taps from per-tap maximum assigned distance; then reduce the corresponding tap costs (discount

factor) and rebuild assignment by greedy matching. This process are meant to make the tap with satellite pins to have a higher priority to be assigned.

Furthermore, as a refinement step, we add a local swap refinement step to improve the assignment. We select the source pins from the high-risk taps (gated by mean/std of max distance), and each source pin considers the top destination taps. A swap is applied only when net gain is positive. After this two steps, we observed a improvement in the average assignment quality (About 2000 in test5, in my local machine).

Code fragment (assignment flow):

```
map<int, int> MFMC::assignPinsToTaps(...) {
    map<int, int> a = assignPinsGreedy(prob, cost_discount);
    a = reassign(a, prob);          // discount-based rebuild
    for (int iter = 0; iter < 8; iter++) {
        map<int, int> b = worstPinSwap(a, prob); // best-partner 2-swap
        if (b == a) break;
        a = b;
    }
    return a;
}
```

Before/after visualization of assignment refinement:

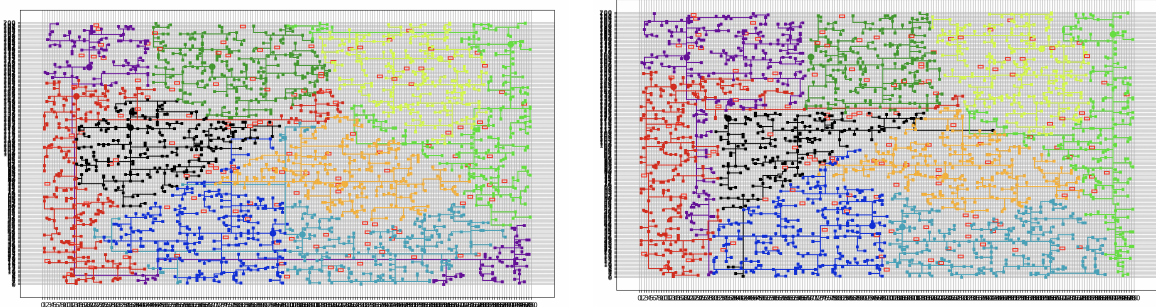


Figure 1: **test4** comparison. Left: before optimization (**v1**). Right: after cost-discount reassignment + swap refinement (**final**).

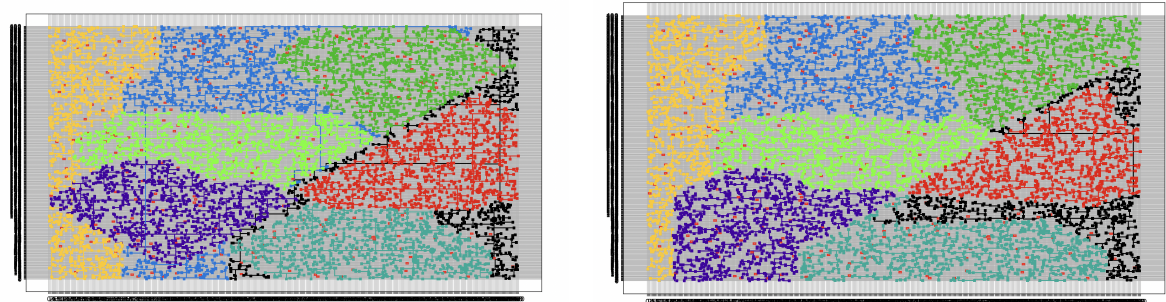


Figure 2: **test5** comparison. Left: before optimization (**v1**). Right: after cost-discount reassignment + swap refinement (**final**). We can see the cluster split problem is reduced.

3.2 A* Routing with Cached Heuristic

The routing stage is implemented in `astar.cpp/hpp`. For each tap, the tree starts from tap location and grows by routing assigned pins one by one to any node already in the current tree.

The A* state uses grid coordinates, with:

- $g(n)$: routed path length from pin start to node n ,
- $h(n)$: heuristic distance from n to the existing tree.

We found that if calculate the closest-tree-distance for each node in each iteration, the runtime will be unacceptably long. So we use a cached closest-tree-distance heuristic variable during search. This cache gives a significant speedup on large cases while preserving stable routing behavior. During expansion, only legal grid edges are explored: blockage rules are checked by the grid model, and edge usage cannot exceed **CAPACITY**.

Code fragment (cached heuristic):

```
int closest_tree_dist = INT_MAX; // cache for heuristic
Point start = {pin.x, pin.y};
closest_tree_dist = heuristic(start, tree_points, closest_tree_dist);
open_set.push(make_pair(closest_tree_dist, start));
```

3.3 Main Retry/Iteration Control (main.cpp)

The top-level control flow in `main.cpp` is:

1. parse input and initialize grid,
2. run assignment stage (`MFMC::assignPinsToTaps`),
3. perform multi-retry routing and keep the best feasible result.

Each retry rebuilds routing from a clean grid and uses diversified orders:

- retry 0: deterministic order (near-first pin order + heavy-tap-first),
- later retries: randomized pin and tap orders.

After each retry, cost is recomputed in a way consistent with the evaluation script: unique routed unit-edge length plus skew term derived from BFS arrival on each tap tree. The retry loop is controlled by a routing-time budget using `chrono`; stopping is adaptive, but the implementation guarantees at least one successful routing retry before budget-based early stop.

Code fragment (time-budgeted retries):

```
const double routing_time_limit_s = 12.0; //tunable for different time budgets
int max_retry = 5; //hard upper bound of retries, also tunable
double est_retry_time_s = -1.0; //estimated cost of next retry
int successful_retries = 0;
for (int retry = 0; retry < max_retry; retry++) {
    double elapsed = chrono::duration<double>(
        chrono::high_resolution_clock::now() - start_routing).count();
    if (successful_retries > 0 &&
        elapsed + safety >= routing_time_limit_s) break;
    if (successful_retries > 0 && est_retry_time_s > 0.0 &&
        elapsed + est_retry_time_s + safety >= routing_time_limit_s) break;
    // ... do one full routing retry ...
    successful_retries++;
    est_retry_time_s = update_estimate(est_retry_time_s, this_retry_time_s);
}
```

4 Experiments

4.1 Setup

- Machine: MacBook Air (13-inch, 2025, M4).
- CPU: Apple M4, 10-core CPU (4 performance + 6 efficiency cores).
- Memory: 16GB unified memory, 120GB/s memory bandwidth.
- Compile command: `g++ -std=c++17 -O2 -Wall *.cpp -o cts`.
- Execute command: `./cts <input_file><output_file>`.

4.2 Quantitative Results

Table 1: Benchmark results

Testcase	Skew	Wirelength	Final Cost	Runtime (s)
test0	2	26	30	0.002
test1	6	48	72	0.002
test2	47	1111	1675	0.049
test3	382	3695	4077	1.436
test4	186	8225	9898	0.998
test5	430	25880	29320	10.62

5 Analysis

Strengths. The method is fast and practical. Greedy assignment gives a good starting point, discount-based reassignment reduces split clusters, and 2-swap refinement further improves local assignment quality. A* routing guarantees legal paths under blockage/capacity constraints, while the cached heuristic and retry randomization improve runtime efficiency and solution robustness.

Limitations. The optimization is still mostly local. Some satellite pins remain when nearby taps are full, and final quality can depend on thresholds (e.g., `kSigma`) in outlier pins and taps detection. In difficult cases, the routing quality may be worse than other dedicated methods.

Overall. This approach provides a fairly good quality/runtime trade-off and scales to larger testcases, but it may still miss better global solutions.